



Full-Stack Development with REMN Stack

CONTENTS

Catalog

01

Introduction to REMN
Stack

02

Front-End and Application
Layer

03

Data Integrity with MySQL

04

Production and Scalability
Considerations



01

Introduction to REM N Stack

Goal: Scalable SaaS applications

01

Building scalable applications

The goal is to create scalable, maintainable, data-reliable applications for SaaS.

02

Maintainable and data-reliable

Ensuring applications are not only scalable but also easy to maintain and have reliable data integrity.

Defining REMN components

REMN Stack definition

REMN Stack is defined as React, Express, MySQL, and Node.js.

Component-based architecture

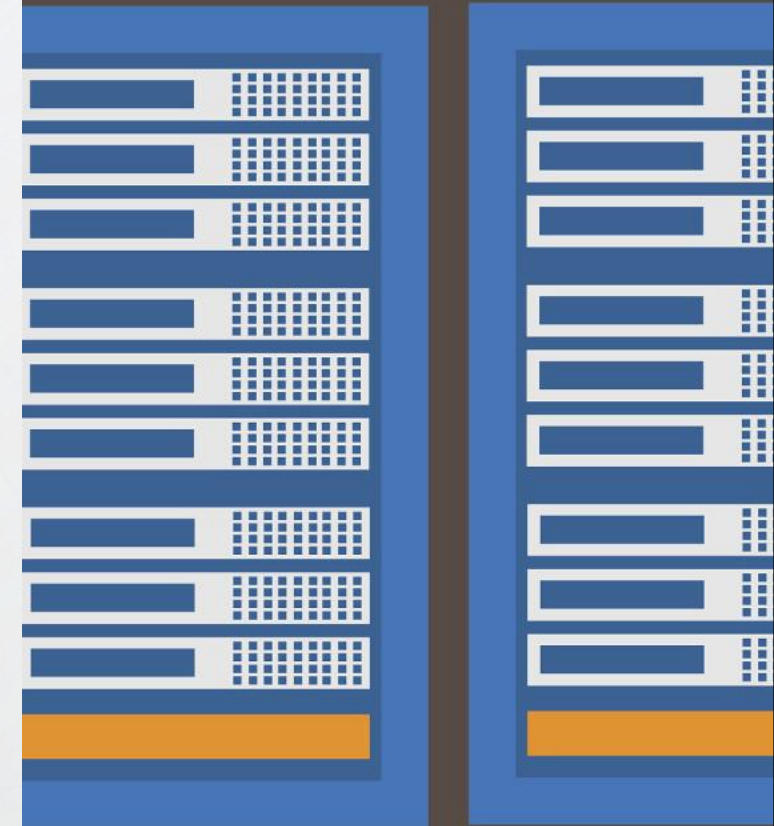
React's architecture is based on components, enhancing code reusability and maintainability.

Asynchronous Node/Express

Node/Express forms the asynchronous core of the system, handling API requests and responses efficiently.

BUILD SERVER

enter



Strategic value of MySQL

Transactional integrity

MySQL ensures transactional integrity for critical user and resource data, a strategic advantage.



Critical user and resource data

MySQL's role in maintaining the reliability of user accounts, billing, and resource tracking is crucial.



02

Front-End and Application Layer

Presentation Layer: React architecture



Component-based architecture

React's modular components enable efficient UI development and state management.



State management with Hooks

Hooks provide a powerful way to manage state and side effects in functional components.



Application Layer: Node/Express

Asynchronous system heart

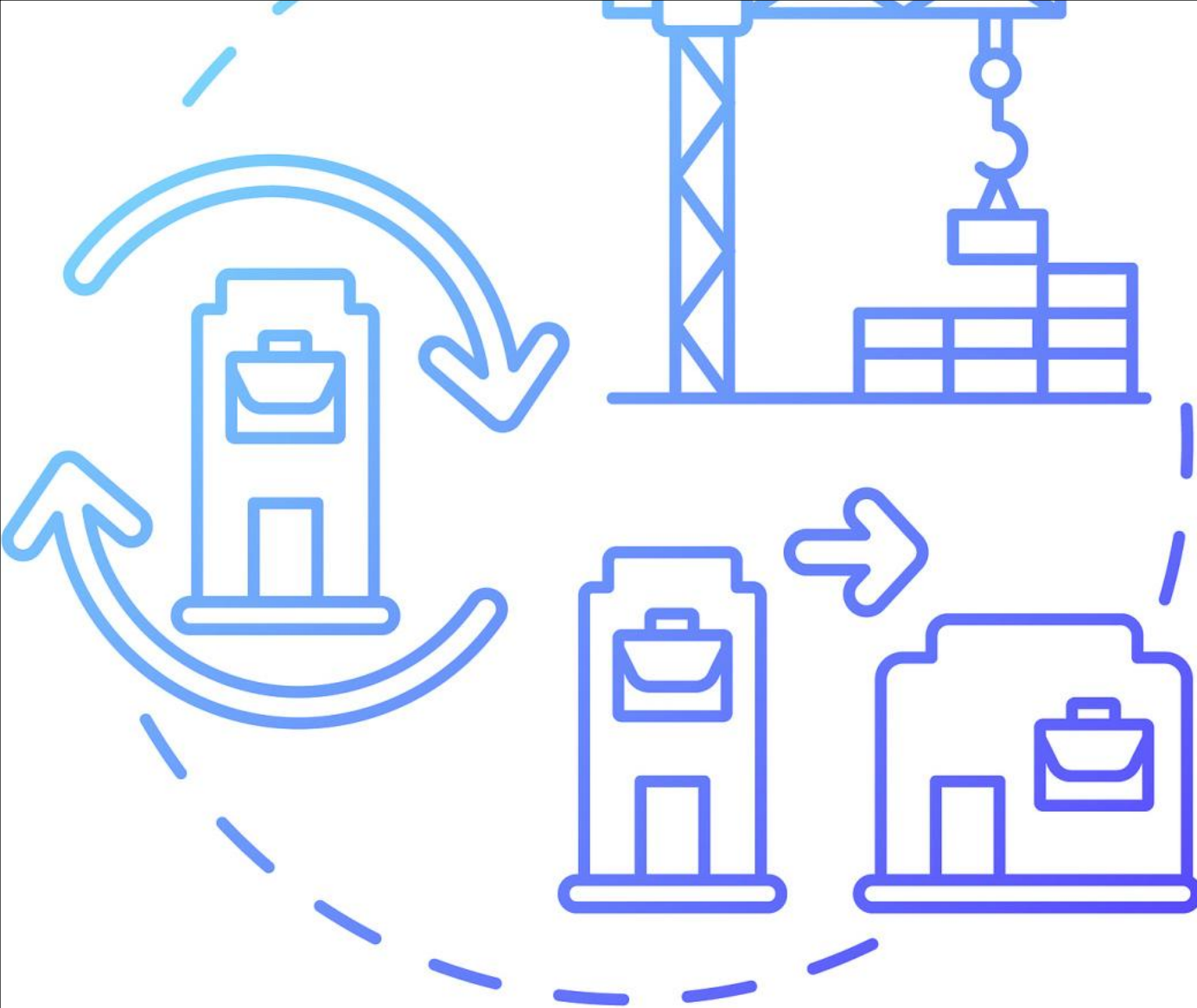
Node/Express handles asynchronous requests, making it ideal for scalable applications.

Building RESTful APIs

Defining clear API contracts with standard HTTP methods (GET, POST, etc.) for interaction.

Middleware for security and parsing

Implementing middleware for authentication, CORS, and data parsing to enhance API security and functionality.



Building RESTful APIs

POSSIBILITY OF

Middleware for security and parsing





03

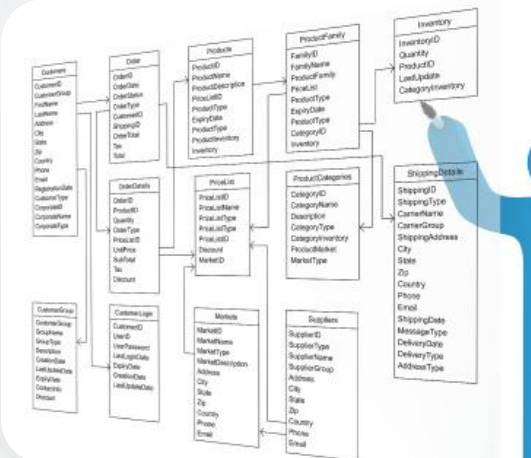
Data Integrity with MySQL

Choosing SQL/MySQL over NoSQL



Strategic Distinction

Choosing SQL/MySQL ensures transactional integrity for critical data.



Critical User and Resource Data

MySQL's relational model is crucial for maintaining user accounts and resource tracking.

ACID compliance for reliability



Guaranteeing Transactional Reliability

ACID compliance is essential for user accounts, billing, and resource tracking.



Critical for User Accounts and Billing

ACID properties ensure the reliability of operations involving user accounts and financial transactions.

Complex relational modeling techniques



Using Foreign Keys and JOINS

Foreign keys and JOINS model complex, stable connections in the database.



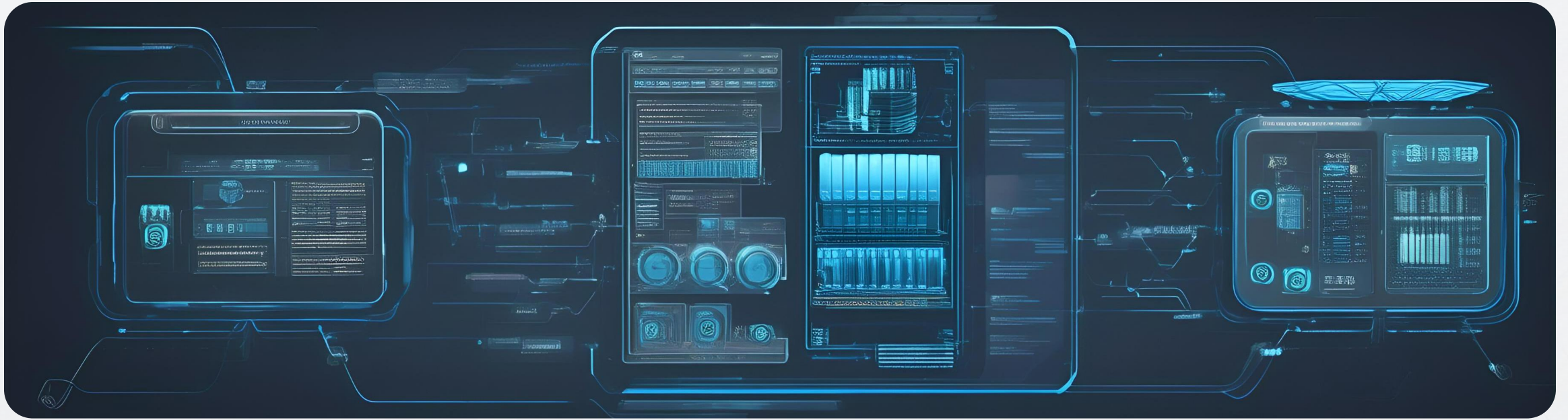
Linking Users to Projects and Models

Example of complex relational modeling: a user linked to a project and to a model version.



04

Production and Scalability Considerations



Professional code structure

Organized Project Structure

Separating client and server code for maintainability and scalability.

Modular Code Design

Implementing modules to enhance code reusability and readability.

Deployment readiness and build process

Environment Variables Setup

Configuring environment variables for different deployment environments.

Efficient Build Process

Streamlining the build process to ensure quick and reliable deployments.

Conclusion: Enterprise-grade SaaS reliability



Relational Stack Mastery

Full stack expertise ensures the reliability needed for enterprise SaaS.

Data-Reliable Applications

The strategic choice of MySQL guarantees transactional integrity for critical data.

THE END

Thank You